

CENTRO UNIVERSITARIO DE OCCIDENTE

División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 1
Primer Semestre 2026

MANUAL TÉCNICO

Práctica 1 – Intérprete de Pseudocódigo con Diagrama de Flujo

202230026 Enrique Alexander Tebalan Hernandez

1. Organización del Proyecto

El proyecto sigue la estructura estándar de un proyecto Android con Gradle, enriquecida con directorios para las especificaciones de JFlex y CUP, y con los JAR de utilidades en la carpeta libs/.

```
practica1/
├── app/
│   ├── build.gradle
│   └── generateParser
│       ├── src/main/
│       │   ├── jflex/Lexer.flex
│       │   ├── cup/parser.cup
│       │   └── java/.../generated/
│       └── (auto)
│           ├── kotlin/.../
│           │   ├── ast/Nodes.kt
│           │   ├── compiler/Compiler.kt
│           │   ├── flowchart/FlowchartView.kt
│           │   ├── reports/Reports.kt
│           │   └── ui/MainActivity.kt
│           └── res/
├── libs/
└── jflex-full-1.9.1.jar
```

← Tareas generateLexer /

← Especificación léxica

← Gramática sintáctica

← Lexer.java, parser.java, sym.java

← Nodos del AST

← Fachada de compilación

← Vista Canvas del diagrama

← Modelos de reportes

← Actividad principal

← JARs de JFlex y CUP

```

├── java-cup-11b.jar
├── java-cup-11b-runtime.jar
├── scripts/
│   ├── setup.sh
│   └── generate.sh
└── build.gradle / settings.gradle

```

← Descarga los JARs
← Regenera fuentes manualmente

2. Análisis Léxico – Gramática para JFlex

El archivo `Lexer.flex` define dos estados: `YYINITIAL` para la sección de pseudocódigo y `CONFIG` para la sección de configuración. El separador `%%%%` cambia de estado.

2.1 Patrones base

Definición	Expresión Regular	Descripción
<code>IntLit</code>	<code>0 [1-9][0-9]*</code>	Entero sin signo
<code>DeclLit</code>	<code>{IntLit}\.[0-9]+</code>	Decimal sin signo
<code>Identifier</code>	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Identificador
<code>StringLit</code>	<code>"[^"]*" '\[^\']*'</code>	Cadena entre comillas
<code>HexColor</code>	<code>H[0-9A-Fa-f]{6}</code>	Color hexadecimal
<code>Comment</code>	<code>#[^\r\n]*</code>	Comentario de una línea
<code>WhiteSpace</code>	<code>[\t\f] LineTerminator</code>	Espacios e interlineados

2.2 Tokens del lenguaje – Sección YYINITIAL

Token	Patrón / Palabra reservada	Descripción
INICIO	"INICIO"	Inicio del algoritmo
FIN	"FIN"	Fin del algoritmo
VAR	"VAR"	Declaración de variable
SI	"SI"	Estructura condicional
ENTONCES	"ENTONCES"	Cuerpo del SI
FINSI / FIN SI	"FINSI" / "FIN SI"	Cierre del SI (ambas formas)
MIENTRAS	"MIENTRAS"	Ciclo mientras
HACER	"HACER"	Cuerpo del MIENTRAS
FINMIENTRAS / FIN MIENTRAS	"FINMIENTRAS" / "FIN MIENTRAS"	Cierre del MIENTRAS
MOSTRAR	"MOSTRAR"	Impresión en consola
LEER	"LEER"	Lectura de variable
ASSIGN	"="	Asignación simple
EQ / NEQ	"==" / "!="	Igualdad / Diferencia
GT / LT	">" / "<"	Mayor / Menor
GTE / LTE	">=" / "<="	Mayor o igual / Menor o igual
AND / OR / NOT	"&&" / " " / "!"	Operadores lógicos
PLUS / MINUS / TIMES / DIVIDE	"+" / "-" / "*" / "/"	Operadores aritméticos
LPAREN / RPAREN	"(" / ")"	Paréntesis
INTEGER	{IntLit}	Literal entera
DECIMAL	{DeclLit}	Literal decimal
STRING	{StringLit}	Literal cadena
ID	{Identifier}	Nombre de variable

2.3 Tokens de configuración – Estado CONFIG

Token	Patrón	Descripción
-------	--------	-------------

%DEFAULT	%DEFAULT	Estilo por defecto
%COLOR_TEXTO_SI	%COLOR_TEXTO_SI	Color texto condicional SI
%COLOR_SI	%COLOR_SI	Color fondo condicional SI
%FIGURA_SI	%FIGURA_SI	Figura del nodo SI
%LETRA_SI	%LETRA_SI	Fuente del nodo SI
%LETRA_SIZE_SI	%LETRA_SIZE_SI	Tamaño letra SI
%COLOR_TEXTO_MIENTRAS	%COLOR_TEXTO_MIENTRAS	Color texto MIENTRAS
%FIGURA_MIENTRAS	%FIGURA_MIENTRAS	Figura nodo MIENTRAS
%FIGURA_BLOQUE	%FIGURA_BLOQUE	Figura nodo BLOQUE
FIGURA_NAME	CIRCULO ROMBO ...	Nombre de figura
LETRA_NAME	ARIAL VERDANA ...	Nombre de fuente
HEX_COLOR	H[0-9A-Fa-f]{6}	Color hexadecimal
PIPE		Separador valor índice
COMMA	,	Separador componentes RGB

3. Análisis Sintáctico – Gramática para CUP

La gramática es LALR(1). A continuación se presenta la gramática en BNF extendida.

3.1 Regla de inicio

```
program ::= algo_section SEPARATOR config_section
```

3.2 Sección de algoritmo

```
algo_section ::= INICIO stmt_list FIN
stmt_list   ::= stmt_list stmt | stmt
stmt        ::= var_decl | assignment | if_stmt | while_stmt
              | show_stmt | read_stmt

var_decl    ::= VAR ID | VAR ID ASSIGN expr
assignment  ::= ID ASSIGN expr

if_stmt     ::= SI LPAREN condition RPAREN ENTONCES block FINSI
              | SI LPAREN condition RPAREN ENTONCES block FIN SI

while_stmt  ::= MIENTRAS LPAREN condition RPAREN HACER block FINMIENTRAS
              | MIENTRAS LPAREN condition RPAREN HACER block FIN MIENTRAS

block       ::= block_list
block_list  ::= block_list block_stmt | block_stmt
block_stmt  ::= var_decl | assignment | show_stmt | read_stmt

show_stmt   ::= MOSTRAR STRING | MOSTRAR expr
read_stmt   ::= LEER ID
```

3.3 Expresiones y condiciones

```
condition ::= expr relop expr
           | condition AND condition
           | condition OR condition
           | NOT condition
           | LPAREN condition RPAREN

expr      ::= expr PLUS term | expr MINUS term | term
term      ::= term TIMES factor | term DIVIDE factor | factor
factor    ::= INTEGER | DECIMAL | ID | LPAREN expr RPAREN | MINUS factor
```

3.4 Sección de configuración

```
config_section ::= config_list
config_list   ::= config_list config_instr | config_instr
config_instr  ::= %INSTRUCCION ASSIGN valor PIPE num_expr

color_value   ::= num_expr COMMA num_expr COMMA num_expr
               | HEX_COLOR

num_expr      ::= num_expr (+|-) num_term | num_term
num_term      ::= num_term (*|/) num_factor | num_factor
num_factor    ::= INTEGER | DECIMAL | LPAREN num_expr RPAREN | MINUS
num_factor
```

3.5 Precedencia de operadores

Nivel (menor – mayor)	Operadores	Asociatividad
1		Izquierda
2	&&	Izquierda
3	!	Derecha (unario)
4	+ -	Izquierda
5	* /	Izquierda
6	- (unario)	Derecha

4. Arquitectura del Sistema

El flujo de compilación sigue el pipeline clásico:

```
Texto fuente
  ↓
Lexer.java (generado por JFlex)
  ↓ Tokens
parser.java (generado por CUP)
  ↓ ProgramNode (AST)
Compiler.kt (walker del AST)
  ↓ Reports + StyleMap
```

FlowchartLayout.kt → FlowchartView (Canvas)
MainActivity.kt → Tabs: Diagrama | Operadores | Control | Errores

4.1 Capas principales

Capa	Clases/Archivos	Responsabilidad
Léxico	Lexer.flex → Lexer.java	Tokenización, errores léxicos
Sintáctico	parser.cup → parser.java, sym.java	Construcción del AST, errores sintácticos
AST	ast/Nodes.kt	Jerarquía de nodos del árbol sintáctico
Compilación	compiler/Compiler.kt	Facade; recorre el AST y genera reportes
Estilos	ast/Nodes.kt:ElementStyle	Almacena configuración visual por índice
Diagrama	flowchart/FlowchartView.kt	Canvas con pan/zoom, dibuja figuras y flechas
Reportes	reports/Reports.kt	Data classes para errores, operadores y control
UI	ui/MainActivity.kt	Editor, botón compilar, TabLayout, RecyclerView

5. Diagrama de Clases (resumen textual)

Por limitaciones de espacio se describe la jerarquía principal:

```
ASTNode
├── StmtNode
│   ├── VarDeclNode (name, init?)
│   ├── AssignNode (name, value)
│   ├── IfNode (condition, body)
│   ├── WhileNode (condition, body)
│   ├── ShowNode (value)
│   └── ReadNode (variable)
├── ExprNode
│   ├── BinOpNode (op, left, right)
│   ├── NegNode (expr)
│   ├── IntLiteralNode / DecliteralNode / StringLiteralNode
│   └── IdNode (name)
└── CondNode
    ├── RelOpNode (op, left, right)
    ├── LogOpNode (op, left, right)
    └── NotNode (cond)

CompileResult (errors, program?, operators, controls, styles)
FlowNode (id, label, cx, cy, width, height, style)
FlowEdge (from, to, label?)
ElementStyle (textColor, bgColor, figura, fontName, fontSize)
```

6. Instrucciones de Compilación y Despliegue

6.1 Pre-requisitos

- Android Studio Panda 2025.3.1 Patch 1
- OpenJDK 25 (o JDK 17+ es suficiente para Gradle)
- Fedora 43 Linux Workstation
- Xiaomi Redmi Note 10S con Android 13 conectado por USB con depuración USB activa

6.2 Primer uso

Ejecutar el script de configuración para descargar los JARs:

Luego abrir la carpeta practica1/ en Android Studio. Gradle detectará las tareas generateLexer y generateParser y las ejecutará automáticamente antes de compilar el proyecto.

6.3 Ciclo de desarrollo

Si se modifica Lexer.flex o parser.cup, Gradle regenera los archivos Java automáticamente en el siguiente build. También se puede regenerar manualmente:

```
cd practica1/scripts
./generate.sh
```

6.4 Ejecutar en dispositivo físico

```
# En Android Studio: Run → Run 'app' (seleccionar el dispositivo físico)
# O desde línea de comandos:
./gradlew installDebug
```

7. Ejemplo de Archivo de Entrada

El archivo de entrada tiene dos secciones separadas por %%%%:

```
INICIO
  VAR a = 10
  VAR b = 20
  SI (a < b) ENTONCES
    MOSTRAR "a es menor que b"
  FINSI
  MIENTRAS (a < 15) HACER
    a = a + 1
    MOSTRAR a
  FINMIENTRAS
  MOSTRAR "Fin del programa"
```

```

FIN
%%
%DEFAULT=1
%COLOR_TEXTO_SI=12, 45-5, 1|1
%FIGURA_MIENTRAS=CIRCULO|1
%DEFAULT=3

```

8. Reportes Generados

8.1 Reporte de operadores matemáticos

Operador	Línea	Columna	Ocurrencia
Suma	3	14	10 + 1
División	5	21	25 / 2
Multipliación	7	10	3 * 4

8.2 Reporte de estructuras de control

Objeto	Línea	Condición
SI	4	a < b
MIENTRAS	7	a < 15

8.3 Reporte de errores

Lexema	Línea	Columna	Tipo	Descripción
\$	2	13	Léxico	Símbolo no existe en el lenguaje
*	3	1	Sintáctico	Se esperaba 'numero'